RAG, Embeddings, and AI Applications

A practical guide to retrieval, agents, evaluation, security, and grounded AI products

Scope and approach

This document is a structured technical learning guide, written to connect foundational ideas to modern practice. It distinguishes established concepts from fast-moving research directions. It is not a systematic literature review and should be read alongside the original papers and documentation in the references.

How to use this guide

Read sequentially for a course-like progression. For each topic, focus on (1) the problem it solves, (2) the mechanism, (3) trade-offs, and (4) how to test whether it works in a real system.

1. Embeddings and semantic retrieval

An embedding model maps text, images, or other inputs into vectors. Similarity measures such as cosine similarity or dot product are used to rank candidates. Embedding quality depends on training objectives, domain, language, chunk length, and whether the model was designed for symmetric similarity or query-to-document retrieval.

Vector similarity is not the same as relevance. A semantically similar passage can fail to answer the question, while a passage with exact keywords may be essential. Retrieval systems often combine dense embeddings with sparse lexical methods such as BM25.

Vector indexes

Exact nearest-neighbor search compares the query against every vector and can be expensive at scale. Approximate nearest-neighbor indexes trade a controlled amount of recall for lower latency. Index configuration, vector normalization, metadata filters, update frequency, and embedding version all affect results.

2. Retrieval-Augmented Generation (RAG)

RAG combines a retriever with a generator. At query time, the system retrieves external passages and places them in the model's context so the response can be grounded in evidence. The original RAG formulation combined parametric language-model knowledge with retrieved documents; production applications commonly use a modular retrieval pipeline.

A standard pipeline

- Ingest: load PDFs, web pages, Markdown, databases, and other approved sources.
- Parse and clean: preserve headings, tables, code, page numbers, and metadata where useful.
- Chunk: split into semantically coherent passages with overlap only when justified.
- Embed and index: generate vectors and store text, metadata, and access controls.
- Retrieve: use query rewriting, metadata filters, dense search, lexical search, or hybrid retrieval.
- Rerank: score the candidate set with a stronger cross-encoder or reranker.
- Construct context: include relevant passages, source identifiers, and clear instructions.
- Generate and cite: answer using evidence and indicate when evidence is insufficient.
- Evaluate and monitor: measure retrieval quality, grounding, latency, cost, and user feedback.

3. Chunking, ranking, and context assembly

Small chunks improve precision but can omit context; large chunks preserve context but may add noise and consume tokens. Fixed-size chunking is easy to implement; recursive splitting respects separators; structure-aware chunking follows headings and document sections; semantic chunking attempts to preserve meaning. The best approach depends on document format and question types.

Hybrid retrieval combines sparse and dense rankings, often using reciprocal rank fusion or a learned combination. A reranker can improve ordering among retrieved candidates but adds latency and cost. Context assembly should remove duplicates, respect document permissions, prioritize direct evidence, and avoid including so much text that the useful information becomes difficult for the model to use.

Citations and provenance

Store document ID, title, URL or file path, page/section, access permissions, ingestion time, and content version. Citations should point to evidence actually used. A model-generated citation that was not verified against retrieved passages is only decorative text.

4. RAG vs fine-tuning vs long context

Approach	Best fit	Main limitation
RAG	Changing facts, private corpora, traceable evidence	Retrieval can miss or mis-rank evidence
Fine-tuning	Style, task behavior, domain patterns	Updates require new training; not a live database
Long context	Small sets of relevant material available directly	Cost and attention dilution; context may be underused
Tool/API call	Current structured facts or actions	Tool failures, permissions, schema and latency

These approaches can be combined. A RAG system may use a fine-tuned model, a long context, and tools. Choose the simplest architecture that satisfies freshness, quality, privacy, cost, and explainability requirements.

5. Agents and tool use

An agentic application uses a model to choose among steps, tools, or actions based on observations. ReAct is a well-known pattern that alternates reasoning-style intermediate steps with actions and observations. Tool use can connect a model to search, calculators, SQL, code execution, calendars, or business APIs.

Do not grant tools unrestricted authority. Define typed schemas, validate arguments, enforce user permissions, limit retries, set budgets and timeouts, and require confirmation for consequential actions. The model should not decide its own access rights. Treat tool output as untrusted input, especially if it contains instructions from retrieved webpages or user-uploaded documents.

6. Evaluation and failure analysis

RAG evaluation must separate retrieval from generation. Retrieval metrics include Recall@k, Precision@k, MRR, and nDCG. Generation metrics include answer correctness, faithfulness to evidence, citation precision, completeness, refusal quality, and user satisfaction. Use human-reviewed examples for ambiguous tasks; automated judges can help scale review but may share model biases.

Failure categories

- Retrieval miss: the correct passage is absent from top-k.
- Ranking failure: evidence is retrieved but buried below irrelevant passages.
- Chunking failure: the passage lacks needed context or splits a table incorrectly.
- Grounding failure: the model ignores evidence or adds unsupported claims.
- Citation failure: cited source does not support the associated claim.
- Permission failure: unauthorized documents leak into retrieval.
- Freshness failure: stale vectors or cached answers survive document updates.

Build a test set of real user questions, expected evidence, acceptable answers, and known unanswerable questions. Evaluate changes against the same versioned set to prevent regressions.

7. Security, privacy, and responsible design

LLM applications face prompt injection, sensitive-data exposure, insecure output handling, excessive agency, vector-store access-control failures, denial-of-service costs, and supply-chain risks. A prompt is not a security boundary. Separate system instructions from retrieved content, but assume a model can still be influenced by hostile text.

- Apply least privilege to every tool, connector, and database query.
- Enforce tenant and document access filters before text reaches the model.
- Redact secrets and minimize personal data in logs.
- Validate generated SQL, HTML, URLs, and tool arguments before execution.
- Use allowlists, quotas, timeouts, monitoring, and human approval for risky actions.
- Test adversarial documents, indirect prompt injection, data exfiltration attempts, and cross-user leakage.

RAG improves access to external evidence; it does not automatically make a model truthful, secure, or compliant.

8. Reference architecture for a study assistant

A study assistant can ingest course PDFs and Markdown notes, preserve headings and page references, chunk and embed them, and store vectors with course/module metadata. A user asks a question; the system authenticates the user, filters documents by ownership and course access, retrieves and reranks passages, builds a compact context, generates a Markdown answer with citations, and records feedback.

Useful product features include streaming responses, source previews, document management, study summaries, flashcards, question generation, “answer only from my notes” mode, and an explicit “not found in the uploaded materials” response. Markdown rendering should sanitize unsafe HTML and code highlighting should avoid executing code.

Operational concerns include background ingestion jobs, file-size limits, OCR for scanned PDFs, duplicate detection, deletion propagation, re-embedding after model changes, audit logs, rate limits, and an evaluation dataset representing actual course questions.

Suggested learning path and exercises

- Build a small RAG prototype over 20 Markdown files and show page/heading citations.
- Compare BM25, dense retrieval, and hybrid retrieval on 30 hand-written questions.
- Create five unanswerable questions and check whether the system admits that evidence is missing.
- Test indirect prompt injection inside a document and verify tools cannot access unauthorized data.
- Track Recall@5, citation precision, latency, token cost, and answer correctness before optimizing.

References and further reading

The references below are starting points. Prefer original papers, official documentation, and careful technical reports. Some papers are historical foundations; others describe techniques whose current implementations may have evolved.

Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. https://arxiv.org/abs/2005.11401

Reimers & Gurevych (2019), Sentence-BERT. https://arxiv.org/abs/1908.10084

Karpukhin et al. (2020), Dense Passage Retrieval for Open-Domain Question Answering. https://arxiv.org/abs/2004.04906

Johnson, Douze & Jégou (2017), Billion-scale similarity search with GPUs. https://arxiv.org/abs/1702.08734

Khattab & Zaharia (2020), ColBERT. https://arxiv.org/abs/2004.12832

Robertson & Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond. https://doi.org/10.1561/1500000019

Yao et al. (2023), ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629

Schick et al. (2023), Toolformer. https://arxiv.org/abs/2302.04761

OWASP, Top 10 for LLM Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/